Why You Must Run on the Latest Release Update

Franck Pachot, Principal Consultant, dbi services

As a consultant, I see too many customers still running on old releases of the Oracle Database: versions which are not supported any longer, or at least which do not have the latest software updates (Security Patch Update (SPU), Patch Set Update (PSU), Release Update (RU) ...) applied. The reason given is that the application software has not been certified, by the vendor, for newer versions of the Oracle Database. And nobody wants to 'take the risk' of running non-certified software for a critical application. I'm convinced that it is a wrong approach and I'll explain how this jeopardises your application more than upgrading the database to newer versions. The new release model of the Oracle Database (with yearly releases such as 18*c*, with RUs and Release Update Revisions (RURs)) will help you to take the right approach, in the hope that the software vendors play the game.

Risks in your environment

Every change comes with a risk, and conservative people try to avoid changes. With no external interaction, a deterministic system will be stable when you don't change anything. But, this is not how it works in IT and the trend is that there are more and more interactions between the systems. You may have a system that has run for years without any problems, and still have the chance to encounter a critical issue. This can happen when a security leak is suddenly exposed through a change in the firewalls, or because a change in the data makes the Optimizer run into another code path which has a 'wrong result' bug. You think that because a system has been running well for years, you are protected. But, unfortunately, it is certain that you will encounter an issue one day. There are bugs in the application. There are bugs in the database. Sometimes we even see systems where a bug in one layer is compensated by a bug in another one, until a little change reveals it.

When a software vendor certifies the application for a specific database version, it does not guarantee that you will have no problem as long as you stay on this version. People often confuse certification and support. Certification is about the tests done in the past, like 'certified on 11.2.0.3', but support is about the future, like 'supported on version >= 11.2.0.4'. The database vendor is in charge of keeping compatibility with previous versions, so that an application developed on one version can run on newer ones. And that's the case with Oracle. The application vendor is in charge of supporting newer versions, even if not certified. And you are in charge of testing the integration in your environment when there is one major change. Only your User Acceptance Testing (UAT), on your data, can certify the application in your environment.

Risks in the database

Even with very limited external interaction you may encounter

a bug after several years. Do you remember the January 2012 CPU/PSU when Oracle published the SCN Flaw? The SCN (System Change Number) is a number that is constantly increasing, like a timestamp to sequence the transactions. This number is essential for all ACID (Atomicity, Consistency, Isolation, and Durability) properties, it is present in all database structures down to each block, and governs the recovery mechanism in case of failure. It can only increase, mostly at Commit, but being coded on 48 bits we should never reach the maximum, as it can go to trillions. But there was a bug (Bug 12371955) in hot backup mode where the SCN was incremented by a large value. And this was even worse when databases were interconnected by database link because each distributed transaction synchronizes the SCNs to the highest value. This means that you can run a database for years and suddenly see it blocked without being able to accept any transaction, and your only solution is to create another database and try to migrate your data into it. If you are running in 11.2.0.2 and didn't apply a PSU after January 2012 then you are exposed. If you are running your database on the 11*g* installation downloaded from the public site http://www.oracle.com/technetwork, without any additional patches, you are exposed because it is the 11.2.0.1 one.

And it is even worse. If you still have old versions where you didn't encounter the problem, and they have database links with newer versions, read MOS Note 2361478.1 and you will see that they may stop functioning in June 2019. An unsupported version is full of time bombs like this: working well and stable for years and then encountering a critical issue.

If you think that you are covered from any blame because you didn't 'take the risk' to go to a version that was not certified by your application software vendor, then think about it.

The day you have the database blocked by a critical issue, or the data corrupted by a data layer bug, or discover that the results were wrong for years because of an Optimizer bug, or the sensitive information exposed by a security leak recently discovered or exposed, then this day, you will be the one responsible for not maintaining the database software to a supported level. For sure, Oracle is very robust and the probability is low. But when you run a version that is unsupported for years, you increase the probability a lot, and you reduce the chances of solving the problem easily. The software vendor certification is not a guarantee here because those bugs were not known at the time the vendor did the certification.

Support and software updates

With Oracle, running the latest version of the software is at no additional cost. You pay each year 22% of the price you paid to buy the licences. This amount covers the support and the software updates. People tend to think that this cost of support is very high because they do not use it as they should do. If you think that this 22% support is only for opening Service Requests (SRs), then for sure it is expensive and not efficient. But there's more. You pay for the knowledge base (known formerly as 'metalink notes') about the issues previously encountered by others, with explanations, workarounds and patches to fix issues. This is the benefit of running one of the major database software packages on the market. Many people run the new releases (or even beta ones) for development, new projects or non-critical applications. They encounter the problems, open an SR and, after a few months, all those critical issues and regressions are known, solved with fixes or workarounds, and the software is stable enough for critical production.

Without any additional cost, you can also proactively avoid those known issues by applying software updates (CPU, SPU, PSU, Proactive Bundle Patches, RUs).

What will you say when you will encounter a security leak? What will be your defence when the issue has been known or fixed for months or years? How will you explain that an update was available to fix this issue, that you paid for the support, but you didn't apply it?

In order to avoid this, or lower the risk, you must apply the security and critical patches proactively, or at least be able to apply the patch as soon as a problem is encountered. And for that, you must run on a supported version of the database. At any time, Oracle provides support for two or three versions: currently 11*g*R2, 12*c*R1 and 12*c*R2 are supported (on 11.2.0.4, 12.1.0.2, 12.2.0.1 and 18.0 patchsets).

Risks in the application

So, you don't upgrade because the application software is not certified for the latest version. What does it mean? It just means that the version of the application you are running on has not been tested on it by the software vendor. The reason is probably that the newer version of the database was not released at the time the vendor tested the software. And, very often, the version that the software vendor certifies is just the version they developed on. If so, this database version is probably already out of support when the application is released and you install it.

But now, let's look at the risks of running the application on a database that is newer than the one that was 'certified'.

Is there any security risk when running on an earlier database software? Of course, you may fear that a new version of the database comes with new features which may introduce a new security issue. But I'm not talking about running the latest release as soon as it is out. Today, you can run 11.2.0.4 or 12.1.0.2, which have had no new features since mid-2013, and apply the latest security patch updates to them. How do you evaluate the risk after four years of security fixes?

On the other hand, if you run on previous versions, such as 10.2, you are running with a security protocol from 2005 (SHA-1, which is considered very unsecured today).

Is there any risk of corruption or a wrong result when running on a version of the database that was not certified by the application? Once again, you can lower the risk by applying the latest bundle patches on a supported version of the database. If you run your application on a 10*g* database, or on 11.2.0.2, you just increase the risk of encountering the bugs that were discovered since the end of support (three years ago). Maybe you already encountered those kind of Optimizer bugs which are qualified as 'wrong result' without knowing it, because those are discovered by chance. The best we can do to lower the risk is to apply the latest fixes without the latest features. And this is possible only in a supported version of the database.

Performance and execution plans

Here is what people fear when upgrading to the newest version of the Oracle Database: at each release, new Optimizer features are implemented. This became even more frequent when some Optimizer changes were introduced in patchsets. And this is where there is a risk: running with an Optimizer that is at a higher release than the one the application performance has been developed and tested on.

But this is not a reason to stop the upgrades. In addition to supporting three current releases (such as 11.2, 12.1 and 12.2 currently), Oracle supports all previous versions of the Optimizer through the `optimizer_features_enable` parameter.

Just have a look at the different versions of the optimizer you can run:

```
SQL> select listagg(VALUE_KSPVLD_VALUES,', ')
     within group (order by ORDINAL_KSPVLD_VALUES)
     from sys.X$KSPVLD_VALUES
     where NAME_KSPVLD_VALUES='optimizer_features_enable';

LISTAGG(VALUE_KSPVLD_VALUES,',')WITHINGROUP(ORDERBYORDINAL_
KSPVLD_VALUES)
--------------------------------------------------------------------
-----------
8.0.0, 8.0.3, 8.0.4, 8.0.5, 8.0.6, 8.0.7, 8.1.0, 8.1.3, 8.1.4,
8.1.5, 8.1.6, 8.1.7
, 9.0.0, 9.0.1, 9.2.0, 9.2.0.8, 10.1.0, 10.1.0.3, 10.1.0.4,
10.1.0.5, 10.2.0.1
, 10.2.0.2, 10.2.0.3, 10.2.0.4, 10.2.0.5, 11.1.0.6, 11.1.0.7,
11.2.0.1, 11.2.0.2
, 11.2.0.3, 11.2.0.4, 12.1.0.1, 12.1.0.2, 12.2.0.1, 12.2.0.1,
18.1.0, 18.1.0.1
```

If your software vendor tells you that its software is certified only for 11.2.0.2 then you can still upgrade to 12.1.0.2 to be on the latest security and critical fixes, and run the Optimizer with the `optimizer_features_enable=11.2.0.2`. You can stay with that for a few weeks: it is a great way to decorrelate the upgrade of the software and then test performance with the new Optimizer features. However, it is better to raise the Optimizer version to the latest as soon as you are confident of the performance. You can use SQL Plan Management to capture and accept the execution plans while you are running with the `optimizer_features_enable` and then you can raise this parameter to the default. When a new feature of the Optimizer comes with a new plan, it will be stored but not used until you accept it. And so you run the latest database software with the behaviour of the application which has been certified – and you get the time to evolve the execution plans without stress.

Release, patchset, updates

You expose your application and data to more risks when you run it on an out-of-support version of the Oracle Database, and this is far more important than the certification of the application. Which version to run, then? Let's take an example. Your application has been certified on Oracle 10.2 but the closest supported version is 11.2, so this is the way to go if you want to stay not too far from the application vendor recommendation. Then you run on 11.2.0.4 which is supported until mid-2021. 'Supported' means that Oracle provides fixes on issues encountered and as soon as a critical issue is encountered, and this includes the most important fixes bundled in quarterly updates.

You may have the feeling that running on supported software requires you to upgrade too frequently. Look at Figure 1: you can see that each version released by Oracle has been supported for more than eight years after the General Availability date.

Eight years is a long time, but, if you wait four years after the release date (because of application certification), and then spend one year in the upgrade project (because the gap with the previous version is very large and you have a lot to test), only three years remain. And you will feel that you need to upgrade frequently to get support.

However, if you don't wait for the application certification, you can start to upgrade your test environments when the version is released and go to production a few months later, when you are sure it is stable, maybe after the first quarterly update.

For sure, you have to test and maybe fix or work around some issues. You have full support to fix those issues because you are on a current version. Then it works, and this is your certification. And your tests are far more relevant than the software vendor tests

Long-term support releases	Release date	End of updates and fixes	
9.2	Jul-2002	Jul-2010	8 years
10.1	Jan-2004	Jan-2012	8 years
10.2	Jul-2005	Jul-2013	8 years
11.1	Aug-2007	Aug-2015	8 years
11.2	Sep-2009	Dec-2020	11 years
12.1	Jun-2013	Jul-2021	8 years
12.2	Mar-2017	Mar-2025	8 years
20.1	*Jan-2020*	*(hypothesis that 20c will be long term)*	*8 years*

FIGURE 1: RELEASE AND END OF SUPPORT DATES

done years ago, because yours were done on your environment, your data and your usage.

Just look at the Oracle release roadmap (MOS note 742060.1). Going back to my example, 11.2.0.4 is the way to go if you want to avoid upgrading again in the next few years because it is the terminal patchset of 11*g*R2. If you're not worried about having a gap between what your software vendor certified and the database version, you can also choose to go to 12.1.0.2, which is the terminal patchset for 12*c*R1. Both provide support for the next two years, with six additional months for 12*c*R1.

Last year, I was involved in several migration projects with customers who had migrated very old databases (9*i*, 10*g*) to consolidate to 12*c*. For most of them we chose the final patchset of 12*c*R1, 12.1.0.2 with the latest Proactive Bundle Patch and the Adaptive Statistics patches, because the decision was taken just after 12*c*R2 was released and at that time there was uncertainty about when support for 12.2.0.1 would end. Currently, you should go to 12.2.0.1 or 18*c*. These databases run old applications which are not certified for 12*c* (and most of them are not even certified for 11*g*). But who cares about certification that was done ten years ago on different data and usage? Our tests had validated that all applications work the same or better than on the previous server. I'm talking about 20 critical production databases where we had only a few queries to fix from the application or through a few SQL baselines. Now, the applications are running efficiently and safely on a supported database.

At other customers, I see applications still running on an old database, because the management do not want to upgrade above the certified version, with all kinds of problems: a smaller choice of hardware, old security protocols, a lot of one-off patches for the bugs encountered ...

We are looking at the old release model here. The release (such as 11.2, 12.1, 12.2) is what is supported in the long term but it needs upgrades (patchsets like 11.2.0.4, 12.1.0.2 or 18.0) during that support window. They are called patchsets but they are actually full installations, with a lot of new features, requiring comprehensive testing and a larger downtime window. In between those upgrades, you apply the smaller updates. If you are concerned only by security fixes, you apply only the SPU. When you want to avoid the risk of outage when encountering a critical bug, you apply the PSU.

The new release model

With the new release model, Oracle wants to provide an easier way to run the Oracle Database in a supported and stable version. The complexity of software is increasing, the exposure to security attacks is rising and the amount of data that is processed is growing. This increases all risks when running an old version of the software. There's no choice: if you don't want to isolate your system, and restrict its usage, you can't run it as you did 20 years ago. The way to secure it is to have the security and critical fixes applied as soon as they are known. The old model had several levels of patches: security fixes (SPU – the former CPU), or critical patches (PSU), or more fixes (Proactive Bundle Patches).

The new model encourages you to have all the latest fixes that are available with the RU.

Release Updates

The RU is the big bundle patch that is released quarterly, which contains the fixes for known problems. The idea is the same as for the Proactive Bundle Patches: the more you have in them, the lower risk you have to apply a one-off patch later. Those RUs should not change the behaviour or the performance of your application. If new features or small enhancements are included, they are limited to those which have no interactions with the other features. If a fix may change an execution plan, it is disabled by default. The idea is that when you encounter the issue, it is easier to enable a fix already present than to apply a one-off. This means that you can apply those RUs regularly without having to test the whole application as you do with releases.

I said 'regularly', but I didn't say 'immediately'. You probably remember the Proactive Bundle Patch last July, which had some problems and was superseded in August. For sure you don't want to risk that on your critical environment. But if you look at the history of bundle patches, this does not happen frequently and it is fixed quickly. Waiting two or three months before applying the latest RU to production should be sufficient to avoid any problems.

The RU is the second number of the version. For example, 18*c* (or 18.0) was available on some platforms in January as 18.1, and you may have the RU to patch it to 18.2 in April, 18.3 in July, etc. The new features which may impact other ones will be included in yearly releases, such as 18*c* for 2018. These will be supported, with RUs released quarterly, for three years. And probably some of them will have longer support (maybe 20*c* will have RUs for eight years).

Release Update Revisions

In the rare case where there is something to fix over the RU, Oracle may provide an RUR, such as 18.3.1 or 18.3.2 on the 18.3 RU. Then, if you choose to wait one or two months, and if there is an RUR, you will apply the RUR instead of the RU. The RUR also includes the latest security fixes so that, even if you wait, you will have the latest with regard to security. However, remember that the goal is to apply the latest RU and not to wait six months and apply an RUR. In any case, RURs will be provided only for the last two RUs (so six months), which means that applying RURs doesn't prevent applying RUs regularly. To put it simply, you cannot use RURs as a replacement for SPUs. There is no replacement for SPUs because applying only security fixes is not considered sufficient to stay in a supported version of the software. You may feel as if you're being forced to update, and being given extra work to apply 18.2, 18.3, 18.4, 18.5 ... but you will appreciate having reduced the gap when you upgrade to 19*c*. Or maybe wait for 20*c* if that becomes the long-term support release. In mid-2020 you may be on RU 18.11 and upgrade to 20.3.

Conclusion

You can continue to run old versions of the database, with the risks that I have described, which are the same whether the software vendor has certified its application or not. But this is not a good idea, and you will be the one responsible when an outage, a security leak or data loss occurs. And, in addition to that, with the new release model, you will not have the latest security fixes, and your data will be at risk. What the new release model brings is the possibility of always being up-to-date: always having the latest security and critical fixes, and being ready to enable other fixes quickly if needed. We will see how the software vendors react. Ideally, they will not certify for one release only (such as 'certified for 19*c*') but just mention the starting version (such as 'supported on 19.2 and higher').

I must add that this new release model was just announced a few months ago and recommendations may change with experience. Note that it is always advisable to follow Oracle's recommendation because Oracle has more information than we do about the number and scope of changes it will bundle in each patch. The best you can do is follow Mike Dietrich's blog (https://mikedietrichde.com/) for up-to-date information. The recommendations may also evolve during the lifecycle of the release, but, in all cases, be sure to always run on a supported version of the Oracle Database. ■

ABOUT THE AUTHOR

Franck Pachot
Senior Consultant, dbi-services

Franck is senior consultant and Oracle technology leader at dbi services in Switzerland. He has 20 years of experience in Oracle databases – all areas from development, data modeling, performance and administration to training. An Oracle Certified Master and Oracle ACE, he tries to leverage knowledge sharing in forums, publications and presentations.

Blog: blog.pachot.net
ch.linkedin.com/in/franckpachot
@FranckPachot